CURRENT-STATE EXPLAINER

AMR Life Expectancy Intelligence — Current-State Explainer

Written for someone with zero background — every term explained, every number sourced, every design choice justified

Grounded in the live repository: the original project brief (`internal/brief/AMR_LIFE_EXPECTANCY_UNIFIED_PROJECT_BRIEF.md`), pipeline run 20260720T144743, `data/published/`, and a direct read of all 43 dashboard route files, as of 2026-07-23. Nothing here is recalled from memory — every claim traces to a specific file.

---

1. The absolute basics — start here if none of this is familiar

What is AMR? AMR stands for **Antimicrobial Resistance**. When a patient gets a bacterial or fungal infection, doctors give them a drug (an antibiotic for bacteria, an antifungal for fungi) that is supposed to kill the microbe or stop it growing. Over time, some microbes evolve so the drug no longer works on them — they've become "resistant" to it. This is a major and growing global health problem: infections that used to be easily treatable can become hard or impossible to treat, and doctors need to know *where* and *for which drugs* this is happening so they can respond.

How do we know a microbe is resistant? Labs take a sample from a sick patient — a swab, blood, urine, whatever the infection site is — and grow the microbe in a dish. They then test it against a panel of drugs. Each test produces a number called the **MIC (Minimum Inhibitory Concentration)**: the lowest amount of the drug that stopped the microbe from growing in that lab test. A low MIC means the drug worked well at a small dose; a high MIC means it took a lot of drug to have any effect, or nothing worked at all.

How does an MIC number become "resistant" or "not resistant"? Medical bodies publish official threshold tables called **breakpoints**. If a microbe's MIC falls below the breakpoint, it's called **Susceptible (S)** — the drug works. Above the breakpoint, it's **Resistant (R)** — the drug likely won't work at normal doses. In between, it's **Intermediate (I)**. For bacteria, this project uses breakpoints published by **EUCAST** (European Committee on Antimicrobial Susceptibility Testing). For fungi, official breakpoints often don't exist yet, so the project instead uses **ECVs (Epidemiological Cutoff Values)** — a softer classification of "Wild-Type" (typical, still-susceptible population) versus "Non-Wild-Type" (atypical, possibly resistant), published by **CLSI** (Clinical and Laboratory Standards Institute).

What's an "isolate"? One single bacteria or fungus sample recovered from one patient specimen. If a lab tests that isolate against 12 different drugs, that produces 12 rows in this project's data — one isolate, many isolate-drug test results.

What's a "country-year"? A lot of this project's analysis happens at the level of "this country, in this year" — e.g. "Kenya, 2019" — because that's the level at which the outcome variable (life expectancy) and most of the external data (funding, vaccination rates, health spending) are reported. Individual patient results get aggregated up to that level for those parts of the analysis.

What does "ISO3" mean? The standard 3-letter code every country has (e.g. `KEN` for Kenya, `USA` for the United States). Different data sources spell country names differently ("Ivory Coast" vs "Côte d'Ivoire" vs "Cote d'Ivoire"), so converting everything to ISO3 codes is how the project makes sure "the same country" from four different datasets is actually recognized as the same country.

Why life expectancy, specifically? This project's central question isn't just "where is resistance highest" — it's "where does resistance actually correspond to *worse health outcomes for the whole population*." Life expectancy (the average number of years a newborn in that country is expected to live, published yearly by WHO and the World Bank for every country) is used as that outcome measure. It's an imperfect proxy — life expectancy is affected by hundreds of things besides AMR — which is why every statistical claim in this project is described as an **association** (things that move together) and never as **causation** (proof that one thing causes the other). That distinction is repeated throughout this document because it is the single most important caveat in the whole project.

---

2. The original idea — what this project set out to do, and why

This project began as a plan written by one team member (Justice), captured in `internal/brief/AMR_LIFE_EXPECTANCY_UNIFIED_PROJECT_BRIEF.md`. Everything built afterward is either a direct implementation of that plan or a deliberate, documented addition on top of it — not a departure from it. This section walks through the original idea in full, in plain language, so you can see exactly what was proposed and compare it to what exists today.

2.1 The original objective

> *"To determine which AMR subtypes — organism-drug-region combinations, characterized by both their current resistance level and their trajectory of change — across bacterial and fungal pathogens are associated with the lowest life expectancy outcomes, and to estimate the relative impact of candidate interventions on closing that gap."*

In plain terms: find which specific combinations of "this bug, resistant to this drug, in this region" line up with countries where people live shorter lives — and then figure out which real-world interventions (more vaccines, better hospital hygiene, more diagnostic testing, and so on) would most plausibly help close that gap.

2.2 The five guiding questions

The brief broke that objective into five specific questions. These are the same five questions (Q1–Q5) that structure this project's results today — nothing was added or dropped:

1. **Q1.** Which resistance profiles — bacterial or fungal — co-occur with the lowest national life expectancy?
2. **Q2.** Is that relationship better explained by antimicrobial overconsumption, weak health-system capacity, low vaccination coverage, or hospital-acquired exposure — and does the answer differ between bacteria and fungi?
3. **Q3.** Where does AMR research-and-development funding concentrate relative to where surveillance shows the heaviest burden — and does that mismatch matter?
4. **Q4.** Which pathogen-drug combinations are not yet high-resistance today, but are trending toward it fast — where would early intervention have outsized value?
5. **Q5.** Which category of intervention, if scaled up in the highest-burden or highest-trajectory countries, would plausibly produce the largest life-expectancy gain?

2.3 The data — the four cohorts, explained fully

A **"cohort"** here just means one surveillance program: an organization that ran its own testing sites, its own years of collection, and its own reporting format, independent of the other three. The project combines four of them, obtained through the Vivli AMR Register under Data Request ID 00013346:

Cohort	Pathogen type	Isolates (raw)	Years covered	What it is
SOAR 201818	Bacterial	2,413	2014–2016	One of three separate SOAR ("Survey of Antibiotic Resistance") surveillance rounds
SOAR 201910	Bacterial	2,318	2015–2018	A second, later SOAR round with its own site list and reporting format
SOAR 207965	Bacterial	3,134	2018–2021	A third, most-recent SOAR round; the only one of the three that flags some results as "not evaluable" (613 rows, excluded)
SENTRY (Vivli/SENTRY antifungal)	Fungal	26,922	2010–2024	A long-running, much larger antifungal surveillance program, contributing the entire fungal side of this project

Together, raw: **2,413 + 2,318 + 3,134 + 26,922 = 34,787 isolates.** After removing 29 isolates that couldn't be matched to a known organism (all from SOAR 207965), **34,758 isolates are classified** — 7,836 bacterial, 26,922 fungal. Every isolate is tested against multiple drugs, which is why the final isolate-drug results table (**343,236 rows**) is much larger than the isolate count — it's isolates × drugs tested, not one row per isolate.

Why combine three separate bacterial rounds instead of using one? Each SOAR round covers different years and, to some extent, different countries and sites. Combining them gives more geographic and temporal coverage than any single round alone — but it also means the pipeline has to reconcile three different spellings of country names, three different ways of writing an MIC value, and three different drug-naming conventions before anything can be compared. That reconciliation work (described in full, stage by stage, in Section 5) is a large share of what the pipeline actually does.

Why is the fungal side entirely one cohort (SENTRY) while bacteria has three? That's simply what data was available and approved for this project through the Vivli Register under this specific data request — there was no second fungal surveillance cohort to combine. This is one of the project's structural limitations, not a choice: it means the fungal analysis rests on a single testing program's methodology and site selection, with no independent cohort to cross-check it against.

2.4 The external data — joined on to give resistance numbers real-world context

Resistance numbers alone don't say anything about life expectancy, funding, or vaccination. The project joins in five outside data sources, matched to each country and year:

Source	What it actually is	Role in the analysis
WHO / World Bank life expectancy	The average number of years a newborn is expected to live, published yearly per country	The outcome variable — what everything else is measured against
ECDC ESAC-Net	A European surveillance network reporting how much antibiotic medicine is consumed per country per year	Tests whether overconsumption of antibiotics tracks with worse outcomes (Europe only — this data source doesn't cover the rest of the world)
WHO/UNICEF vaccination coverage (Hib3, PCV)	The percentage of children vaccinated against Haemophilus influenzae type b and pneumococcal	Tests whether vaccination coverage tracks with lower resistance-linked mortality

Source	What it actually is	Role in the analysis
	disease — two vaccines that reduce the bacterial infections this project studies	
World Bank health-system indicators	Health spending as a percent of GDP, and hospital beds per 1,000 people	A "does the country have the health infrastructure to cope" control variable
GBD 2023 (Global Burden of Disease study) — Socio-demographic Index (SDI) and LRI	A composite score published by the Institute for Health Metrics and Evaluation (IHME) summarizing a country's income, education, and fertility rate, plus a separate estimate of lower-respiratory-infection burden	A broad proxy for overall national development level — turns out to be the single strongest predictor in the fungal model (Section 3.5)
Global AMR R&D Hub funding data	A public database tracking global research-and-development investment specifically targeted at antimicrobial resistance	The funding side of Q3 — compared against where surveillance shows the actual disease burden

2.5 The original planned method, step by step

The brief specified this sequence, executed after the four cohorts are cleaned and combined:

1. **Descriptive profiling** — basic resistance/susceptibility rates broken down by organism, drug class, country, year.
2. **Evolutionary layer** — an "Evolutionary Fitness Score" and "Distance-to-Failure," estimated from how MIC values shift over time in the handful of countries with enough repeated testing to show a trend.
3. **Clustering** — grouping organism-drug-country combinations into typologies based on how high their current burden is and how fast it's rising, done separately for bacteria and fungi since they behave differently.
4. **External data join** — attaching life expectancy, consumption, vaccination, and health-system data to each country-year.
5. **Association analysis** — a statistical regression of life expectancy on resistance burden, trajectory, consumption, vaccination, and health capacity.
6. **R&D alignment** — comparing Hub funding to surveillance burden.
7. **Intervention impact** — scoring plausible life-expectancy gains for different intervention categories (vaccination, antibiotic stewardship, diagnostics, R&D, water/sanitation/infection-prevention for bacteria; antifungal stewardship, diagnostics, infection-prevention, and R&D for fungi — fungi has no licensed vaccine, so "vaccination" isn't a candidate category on that side by design, not by omission).

Every one of these seven steps exists in the pipeline today, in this order, as specific stages (Section 5 shows exactly where).

2.6 Hard constraints the brief itself flagged from the start

The original brief was upfront about limits before any code was written — these aren't problems discovered later, they were anticipated:

- Only **6 countries** (Ukraine, Turkey, Tunisia, Pakistan, Kuwait, Vietnam) have enough repeated bacterial testing over time to support a genuine trend estimate.
- Only **7 countries** (Italy, Spain, Turkey, Chile, Czech Republic, Argentina, Greece) have enough overlap to compare bacteria and fungi side by side in the same place.
- More than **48,000 fungal rows** lack any published classification standard for the drug in question — for those, the honest answer is a range, not a single number (this is exactly what Section 3 explains in depth).

- The life-expectancy regression was always intended to produce a **suggestive association, not causal proof** — the brief itself states this, it isn't a caveat added after the fact.
- The European-only antibiotic consumption data is thinner for fungi than bacteria, and the Hub funding data excludes private/venture-capital investment by design (only publicly tracked R&D funding is counted).

---

3. Why an "integrity layer" was added on top of the original idea

This is the answer to "why doesn't the final product look exactly like a simple read-through of the five steps above" — and it's the single most important design decision in the whole project, so it gets its own section.

3.1 The problem, explained with a plain example

Imagine a country reports "500 bacterial samples tested, 100 resistant" for a drug — 20%. Now imagine another country reports the same 20% number, but their lab only recorded *whether a resistance gene was detected at all*, not what fraction of tested bacteria actually carried it. A viewer looking at both numbers side by side would assume they mean the same thing. They don't — and treating them as if they did would silently overstate confidence in the second number.

This happens in this project's real data in two concrete ways: - **Detection-only genotype fields.** For certain beta-lactamase resistance genes in the SOAR data (and, at a larger scale, in the ATLAS dataset used to stress-test the method), the lab only recorded "gene detected: yes/no" for the isolates that were tested for it — not a controlled, randomly-sampled prevalence estimate across the whole population. Reporting that as if it were "X% of all bacteria in this country carry this gene" would be a real overstatement of what the data can support. - **Breakpoint-absent fungal pairs.** For a large share of the fungal data (roughly 1 in 5 of all fungal isolate-drug rows), no official CLSI breakpoint or ECV exists yet for that specific fungus-drug combination — so the question "resistant or not?" genuinely has no defined answer. Forcing a susceptible/resistant call here wouldn't be measuring anything real.

The original brief already anticipated this — its own §5 (Step 8) and §4.4 already called for "fixing identifiability at the data layer before analysis." What the team added on top (documented as the brief's own §2) is the specific mechanism for doing that consistently across the whole pipeline, plus a way to prove — not just assert — that the fix works.

3.2 The mechanism: a "quality gate" on every published row

Every row of every result the project publishes carries a label called `quality_gate`, with exactly three possible values:

- `pass` — the sample size is large enough, and the classification is unambiguous enough, that a specific number or rank can be published with confidence.
- `bounds_only` — the data can't support one single confident number, but it can support a *range*. This uses a statistical method called **Manski bounds** (after economist Charles Manski, 1989): instead of guessing a single value between the best-case and worst-case scenario, the method calculates the *tightest range consistent with everything actually observed*, and publishes that range instead of pretending to know the exact midpoint. A real example from this project: for one detection-only genotype pilot dataset, instead of asserting "the prevalence is 61.8%," the bound published is **[55.1%, 68.5%]** — an honest "somewhere in here, and here's exactly how we know that."

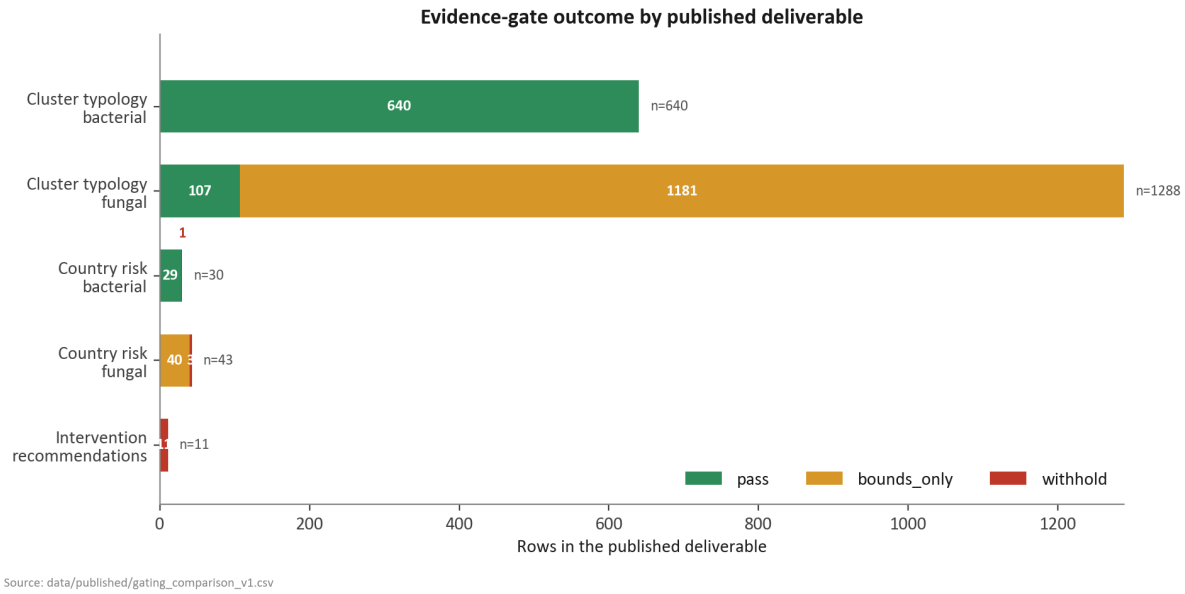
- `withhold` — nothing publishable yet. The row still shows up in the project's internal ledger, so you can see that the question was asked and *why* it couldn't be answered — but no score, rank, or number is shown publicly for it.

Why not just drop the unpublishable rows silently instead of a three-way label? Because silently dropping a row hides the gap entirely — a viewer would have no way of knowing a question was even asked. Keeping every row visible, labeled with *why* it's withheld, is what lets someone audit the project's honesty rather than just trust it.

3.3 What this produces in the actual published results today

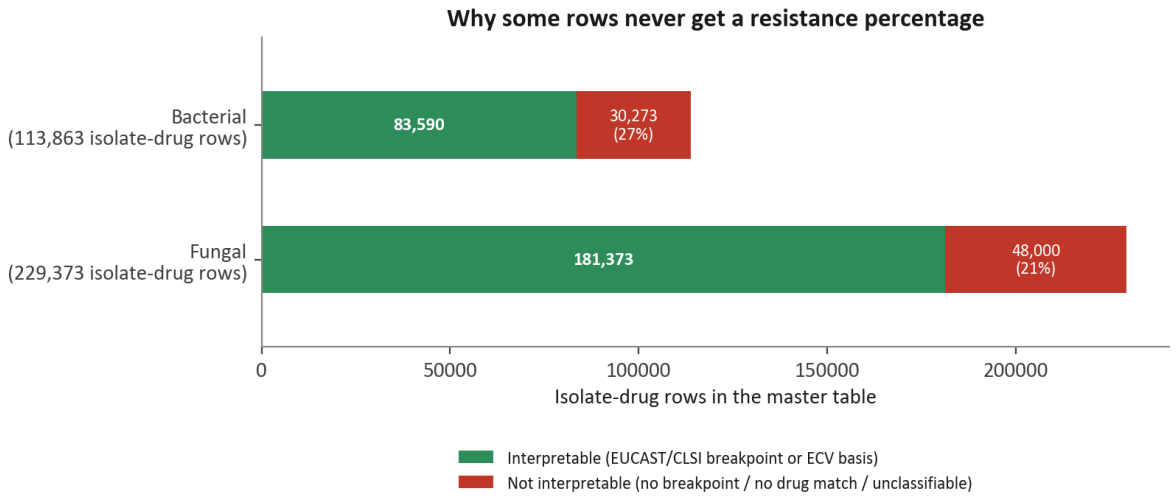
- Bacterial country risk ranking: **29 of 30** countries pass — this pathogen type has enough data to support a real ranking, with just one country (Ghana) withheld for having too few years of data.
- Fungal country risk ranking: **0 of 43** countries pass — the fungal side of the data genuinely does not support a confident country ranking today, and the project says so rather than inventing one.
- All **11** candidate intervention rows are withheld — none of the intervention categories currently clear the bar for a public "this would gain you X years of life expectancy" claim. The dashboard's policy page shows an empty ranked list, *with a stated reason for each row*, rather than forcing a ranking the data can't support.

The chart below shows this outcome for every major published result today:



Source: data/published/gating_comparison_v1.csv

And this shows *why* individual rows end up unable to receive a resistance percentage in the first place — this is the classification-basis split that feeds directly into the gate above:



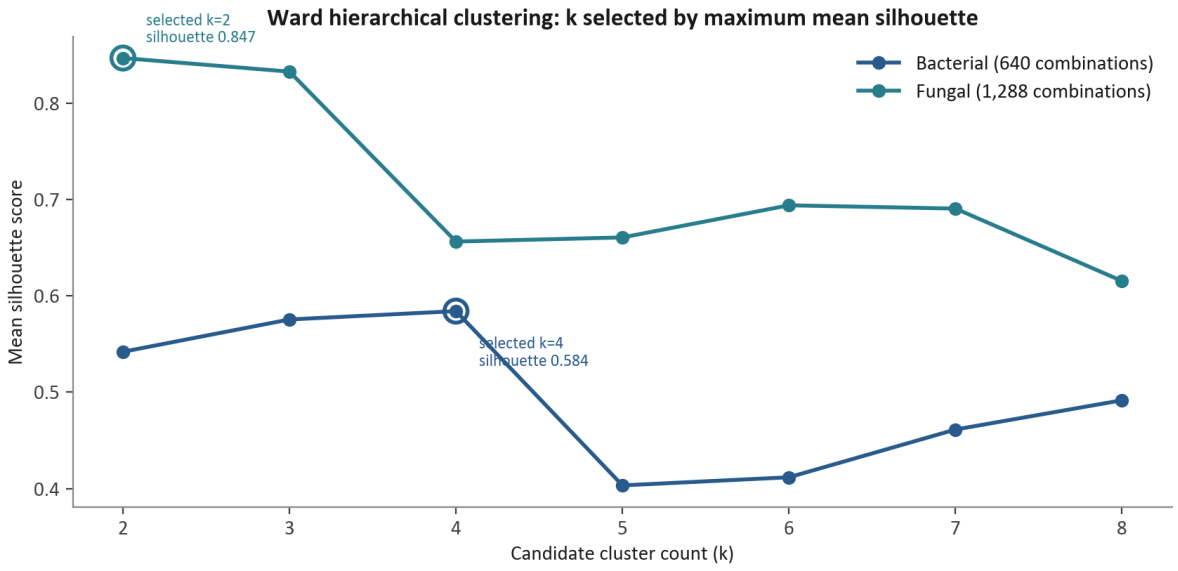
Source: data/published/identifiability_ledger_v1.csv (classification_basis breakdown)

3.4 A companion piece: turning "we don't know" into "here's what would help"

Rather than stopping at "this can't be published yet," the project includes a **budget allocator** (`evidence_gate_core/allocator.py`) that answers a follow-up question: if you had a fixed amount of additional lab-testing budget, which specific gap would it be most valuable to close first? This turns a withheld result into an actionable recommendation instead of a dead end.

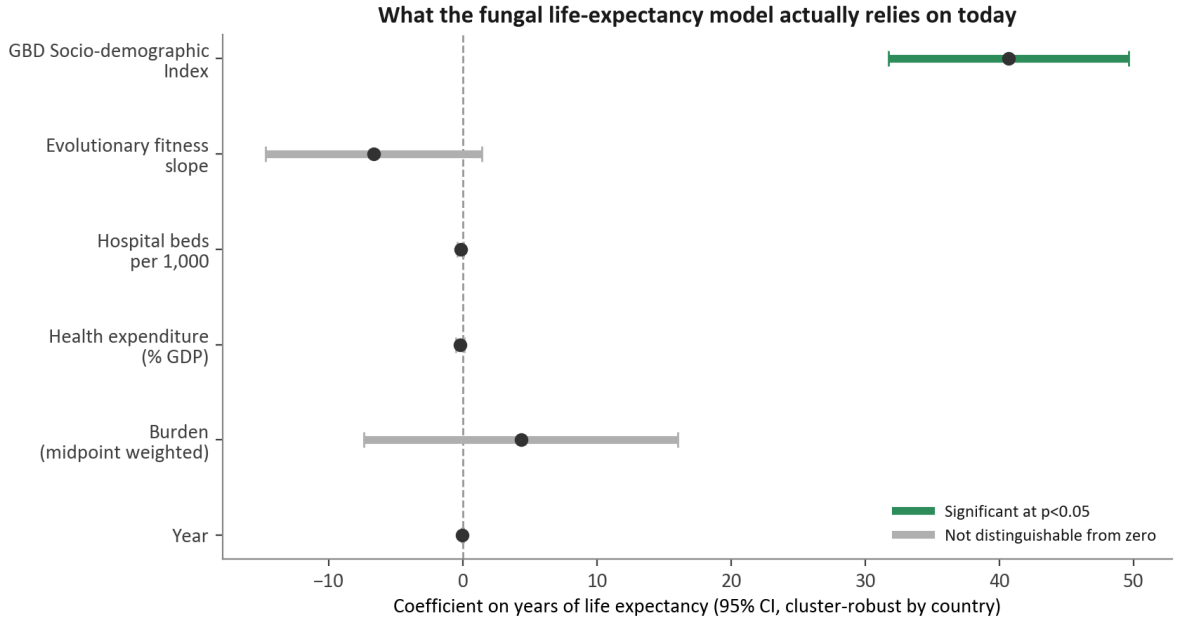
3.5 Two statistical methods used to keep the "pass" numbers honest

How groups are decided (clustering). When the project sorts every organism-drug-country combination into a typology (e.g. "moderate," "high burden," "high trajectory"), it doesn't pick the number of groups in advance. It tries every candidate group count from 2 to 8, measures how cleanly each one separates the data (a standard statistic called a **silhouette score** — higher means cleaner, more distinct groups), and picks whichever count scores best. Bacteria and fungi are scored separately because they don't behave the same way — and indeed they land on different answers: **4 groups for bacteria, 2 for fungi.**



Source: analysis/bounds/cluster_diagnostics_v1.csv

How statistical confidence is measured (cluster-robust standard errors). The life-expectancy regression uses several years of data per country, not just one. Treating each country-year as if it were a totally independent, unrelated data point would understate how uncertain the results really are, because multiple years from the same country tend to be similar to each other — that's called **within-country correlation**, and ignoring it makes a result look more statistically confident than it actually is. The project instead uses **country-clustered standard errors**, the standard statistical correction for exactly this situation. Applying that correction honestly is what determines which factors in the model actually hold up:



Source: analysis/bounds/association_ols_coefficients_v1.csv (fungal model, n=269 country-years, 36 countries)

Reading this chart: the dot is the estimated effect of that factor on life expectancy; the horizontal line through it is the 95% confidence interval (the range the true value plausibly falls in). A green line that doesn't cross the dashed zero line means the project is confident that factor has a real, non-zero relationship with life expectancy. A grey line that does cross zero means the data cannot currently distinguish that factor's effect from having no effect at all.

Today, only **GBD Socio-demographic Index** clears that bar in the fungal model — removing it from the model collapses its explanatory power (R^2, a 0-to-1 measure of how much of the variation in life expectancy the model explains) from 0.638 down to 0.175. Health spending and hospital-bed capacity, despite sounding like they should matter, are not statistically distinguishable from zero once this correction is applied. This is reported as-is, not smoothed over, because the model is explicitly presented as showing **association, never causation** — a pattern in the data, not proof of what causes what.

---

4. Architecture — how the pieces fit together

```
Vivli Register CSVs (private, never leave analysis/)
        │
        ▼
  analysis/run_all.py  →  30-stage pipeline (Section 5)
        │
        ├─ writes runs/<run_id>/pipeline_run_manifest_v1.json
        │
        ▼
  data/published/   (gated CSVs + dashboard_bundle_v1.json — public, on GitHub)
        │
        ▼
  dashboard/public/data/published/   (mirror copy, served to the browser)
        │
        ▼
  React dashboard fetches dashboard_bundle_v1.json client-side, once, on load
        │
        ▼
  39 routes render KPIs / maps / tables / charts from that one JSON file
```

A quick vocabulary note before this makes sense: a **pipeline** here just means a sequence of Python scripts that run one after another, each one reading the previous step's output and producing new files. A **JSON file** is a plain text file that stores structured data (numbers, labels, nested lists) in a format both humans and computer programs can read — it's the file format the pipeline's final output is packaged into. **React** is the programming framework the dashboard's website is built with. An **API** normally means a live connection where a website asks a server a question and gets a fresh answer back in real time — this project deliberately does *not* use one for its AMR data (explained next).

Why a static JSON file instead of a live database or API? The dashboard has no backend server of its own for AMR data — a separate tool called Supabase exists in this codebase, but it's only used for login accounts, an admin console, and a few unrelated utility features (Section 8). The actual AMR content is entirely files the pipeline writes to disk. Practical effects of this choice: the dashboard works even with no internet connection to a live database once the file is loaded; there's no database to keep in sync with the Register data (which Vivli's data-use terms don't allow to leave the pipeline anyway); and every number shown on the dashboard can be traced back to one specific file a reviewer could open directly, rather than trusting an opaque database query. The tradeoff: nothing on the dashboard is more current than the last time someone re-ran the pipeline and republished. That's why a footer reading `Pipeline {run_id} · bundle {generated_at}` appears on almost every page — so a viewer always knows exactly how fresh the data is, rather than assuming it's live.

Why both gated CSVs and a bundle, rather than one or the other? The gated CSVs (spreadsheet files ending `_gated_v1.csv`) are the audit trail — the same format a statistician or judge would open directly, with the `quality_gate` label spelled out per row. The bundle is the same information pre-packaged into one dashboard-shaped JSON file, so the website doesn't have to download and stitch together nine separate spreadsheets every time someone opens a page. Both come from the same pipeline run, so they can't drift apart from each other.

---

5. The pipeline, step by step — every stage, from raw files to final numbers

This section is the complete, ground-truth walkthrough for anyone who wants to know exactly what happens to the data between "four raw surveillance files" and "a number on the dashboard." It follows the exact order encoded in the pipeline's own source of truth, `analysis/src/pipeline_registry.py`, which lists all 30 stages of a full run. Nothing here is summarized from memory — every input file, exclusion count, and output file name below is read directly from the pipeline's own code and its own generated logs.

Three design rules apply to every single stage below, stated once here instead of thirty times: **(1) fail fast** — each stage runs a Check at the end, and the whole pipeline stops immediately if it fails, so a broken stage can never silently pass bad data downstream; **(2) log every exclusion** — nothing is ever silently dropped; every isolate or row removed from the working data gets a line in an exceptions log explaining why; **(3) never fabricate** — if a value can't be determined (a missing breakpoint, an unresolved drug code), the pipeline records an explicit "we don't know" reason instead of guessing a number.

5.1 Phase 1 — Preprocessing: turning four raw files into one clean table (13 stages)

This phase's entire job is: take the three SOAR bacterial files and the one SENTRY fungal file exactly as Vivli delivered them, and produce one single, consistent table where "Kenya" always means the same thing, an MIC value always means the same thing, and a resistance/susceptibility call always means the same thing, no matter which of the four original files a row came from.

Step 1 — Country harmonization (`step01_country.py`). *Goes in:* every raw country string across all four cohorts. *What happens:* each string is looked up in a crosswalk table and mapped to its 3-letter ISO3 code. *What gets resolved:* 59 distinct raw spellings collapse to 57 ISO3 codes — two are genuine, reviewed merges rather than errors (`"UK"` / `"Scotland"` → `GBR`; `"Slovak Republic"` / `"Slovakia"` → `SVK`). *Comes out:* `crosswalks/country_iso3_crosswalk_v1.csv`, consumed by every later step that needs a country.

Step 2 — Date and year parsing (`step02_date.py`). *Goes in:* raw collection-date fields, which arrive in at least three different formats depending on cohort (real datetimes, text strings like `"15-Dec-16"`, and plain integers). *What happens:* each format is parsed according to its actual type — critically, a plain integer is only ever treated as a literal year, never accidentally reinterpreted as an Excel date-serial number (a classic spreadsheet bug this step explicitly guards against). *What gets left behind:* any date that can't be parsed, or that falls outside the valid range (2000–2025) or outside that specific cohort's documented collection window, is logged to `exceptions/date_parse_exceptions_log_v1.csv` — zero on the current clean run. *Comes out:* a parsed year for every isolate.

Step 3 — Organism harmonization (`step03_organism.py`). *Goes in:* raw organism-name strings, bacterial cohorts only (SENTRY's fungal species names pass through untouched until Step 10). *What happens:* every string is mapped to one canonical species name. *What gets excluded:* 29 isolates — all from SOAR 207965 — for being no-growth results, environmental contaminants, or fungal organisms that turned up inside a nominally-bacterial file. These 29

are removed from the analysis-ready data but never deleted from the record: they're named individually in `exceptions/organism_exclusions_log_v1.csv`. *Comes out:* `crosswalks/organism_crosswalk_v1.csv`.

Step 4 — Drug code crosswalk (`step04_drug.py`). *Goes in:* every drug column name or code across all four files — labs abbreviate drug names differently, and different cohorts use different column headers for the same drug. *What happens:* each raw identifier maps to one canonical drug name. Two specific judgment calls are made and documented rather than hidden: the code `CDN` is mapped to cefdinir but flagged `resolution_status=provisional` (strong internal evidence, not confirmed against an official data dictionary); the code `DIN` cannot be confidently resolved at all and is mapped to `UNRESOLVED`, explicitly flagged for exclusion from any cross-cohort drug comparison. SOAR 207965 also tests amoxicillin/clavulanate at two different dosing strengths, and both are kept as separate `dosing_variant` values rather than merged into one. *Comes out:* `crosswalks/drug_code_crosswalk_v1.csv`.

Step 5 — MIC notation normalization (`step05_mic.py`). *Goes in:* raw MIC text values from the three SOAR files only (SENTRY's MIC columns already arrive as plain resolved numbers, with no comparator symbol to parse). *What happens:* strings like `<=0.06`, `</= 0.06`, or `<0.008` are split into a canonical comparator (`<=`, `>`, or `=`) and a numeric value placed on the standard log2 dilution scale used throughout microbiology. *Comes out:* `master/mic_parsed_values_v1.csv` (113,863 parsed bacterial MIC cells), plus a per-cohort table of which drug-dilution combinations were actually tested (`crosswalks/soar_drug_dilution_panel_v1.csv`), plus logs of any parse failures or dilution-series violations (both empty on the current clean run).

Step 6 — Evaluability filtering (`step06_evaluability.py`). *Goes in:* SOAR 207965's own `Evaluable` quality flag — a lab-assigned marker unique to that one cohort; the other three cohorts don't have this column at all. *What happens:* every isolate marked `Evaluable=N` (613 of them, about 19.6% of that cohort) is documented. *What gets left behind:* these 613 isolates are excluded from the master table and from every resistance-rate denominator later in the pipeline — but they still exist in `isolate_registry_v1.csv` and in `exceptions/evaluability_exclusions_log_v1.csv`, so the exclusion is fully auditable rather than invisible. A companion stage later in the run, **Step 6 (rates)** (`step06_evaluability_rates.py`), runs after the master table exists and measures exactly how much this exclusion shifted each drug's resistance-rate denominator.

Step 7 — Resistance classification (`eucast_breakpoints.py` + `step07_classification.py`). This is the step that actually turns a raw MIC number into a medical call. It runs in two halves because bacteria and fungi use fundamentally different classification standards (Section 1 explained why). *Bacterial half:* parses the official EUCAST breakpoint tables — version 8.1 for the two older SOAR cohorts, version 10.0 for the newest one — and classifies each isolate-drug pair as Susceptible, Intermediate, or Resistant, or assigns one of several documented non-result reasons (no breakpoint published for that organism-drug pair, EUCAST's own "not recommended" footnote, etc.) rather than guessing. *Fungal half:* a three-tier hierarchy — first check whether SENTRY already supplied a pre-computed CLSI category; if not, look up a published ECV to classify Wild-Type vs Non-Wild-Type; if neither exists, mark the row unclassifiable. ECV coverage is real but narrow — only about 9 of SENTRY's roughly 200 raw fungal species have a published ECV at all. *Comes out:* the `resistance_category` and `classification_basis` columns that appear on every single row of the final master table — `classification_basis` is never left blank, so a reader can always tell exactly how (or whether) a call was made.

Step 8 — Beta-lactamase bounds (`step08_beta_lactamase_bounds.py`). *Goes in:* the bacterial beta-lactamase gene test result columns, which record `POS` / `NEG` /blank — and a blank here means "not tested," not "negative," exactly the kind of detection-only field Section 3 warned about. *What happens:* rather than reporting a raw positivity percentage that would implicitly treat blanks as negatives, this step computes a Manski bound — a range — for beta-lactamase prevalence, split by organism × country × year. *Comes out:*

`bounds/beta_lactamase_bounds_v1.csv`. This is the first of several places the integrity-layer logic (Section 3) is actually load-bearing inside the core pipeline itself, not bolted on afterward.

Step 9 — Age harmonization (`step09_age.py`). *Goes in:* SOAR's continuous patient-age values. *What happens:* ages are binned into four age bands (`0–17`, `18–30`, `31–60`, `61+`) that match the age-band categories SENTRY already reports natively, so bacterial and fungal data can be compared on the same age groupings. The original continuous age is kept alongside the band for bacteria-only analyses that want finer resolution. Any impossible negative age is logged and excluded (`exceptions/age_sentinel_exclusions_log_v1.csv`).

Step 10 — Deduplication and master assembly (`step10_master.py`). The final and largest step of Phase 1, in two parts. *Part A — checking for duplicate isolates:* the three SOAR cohorts' collection windows overlap at two specific points (Vietnam in 2018, Ukraine in 2016), raising the possibility the same patient's sample was submitted to more than one SOAR round. The step builds a "fingerprint" per isolate (country, year, organism, age, and MIC values across the 13 drugs common to all three SOAR cohorts) and flags matching fingerprints as candidate duplicates — but never auto-deletes them; every candidate is written to `exceptions/dedup_review_log_v1.csv` for a human to review. *Part B — building the master table:* re-applies every one of Steps 1–9's transforms and re-enforces every exclusion rule (Evaluable=N, organism exclusions, zero-measurement isolates) to assemble two final files: `master/master_table_v1.csv` (343,236 rows — one row per isolate-drug test pair, 22 columns) and `master/isolate_registry_v1.csv` (34,787 rows — every raw isolate, including the excluded ones, with an explicit `exclusion_reason` column). *Immediately after:* `pipeline_acceptance_check.py` independently re-verifies that every number reconciles — it does not trust Step 10's own self-report, it recomputes.

The exact accounting, so nothing is unaccounted for: for every cohort, `raw isolates = isolates that made it into the master table + organism exclusions + Evaluable=N exclusions + isolates with zero drug measurements + duplicates removed (always 0 – candidates are logged, never auto-removed)`. For SOAR 207965 specifically: **3,134 = 2,521 + 29 + 584 + 0.** This same equation is checked for all four cohorts before the pipeline is allowed to continue.

What Phase 1 leaves behind, in total: 613 Evaluable=N isolates (excluded from denominators, never deleted), 29 organism-exclusion isolates, and 0 date-parse or MIC-parse failures on the current data — none of it deleted from the repository, all of it named in a specific exceptions log.

5.2 Phase 2 — Integrity layer: stress-testing the method on two more datasets (5 stages)

Before the pipeline trusts its own quality-gate mechanism (Section 3) enough to apply it to the four main cohorts, it first proves that mechanism works on data specifically chosen because it's messier and larger than SOAR/SENTRY. This phase pulls in **two additional datasets that are not part of the four core cohorts and never appear in the master table**: an extract from the **ATLAS** surveillance database focused on *Klebsiella pneumoniae* carbapenemase-gene results, and a dataset called **PLEA**. Both exist in this pipeline for one purpose only — to calibrate and stress-test the integrity method — not to expand the core AMR findings.

Step 19a — Clean ATLAS + PLEA (`step19a_clean_evidence_gate_cohorts.py`). *Goes in:* the raw ATLAS and PLEA extracts. *What happens:* each is run through its own cleaning routine and written to a dedicated "clean store," with a manifest recording exactly how many rows each produced. *Check:* the pipeline hard-fails if either output file is missing or empty.

Step 19 — ATLAS bounds + identifiability table (`step19_evidence_gate_bounds.py`). *What happens:* computes Manski bounds for carbapenemase-gene positivity in the ATLAS Kp data, both globally and specifically for Sub-Saharan Africa. This step also contains a concrete, previously-fixed data-quality bug worth naming as a real example of the pipeline catching its own mistakes: the shared country-crosswalk table was originally built only for the

SOAR/SENTRY cohorts' country-name vocabulary, and didn't recognize six real Sub-Saharan African country spellings used in the ATLAS export (Cameroon, Ivory Coast, Uganda, Malawi, Namibia, Mauritius) — silently dropping real isolates from the Sub-Saharan Africa bound with no warning. The fix, visible directly in the current code, adds those six spellings as an explicit alias table before the lookup runs. *Comes out:* `bounds/identifiability_bounds_v1.csv`, the unified Manski-bounds table spanning both the ATLAS calibration data and the SOAR/SENTRY beta-lactamase bounds from Step 8.

Step 20 — Sampling validation (`step20_sampling_validation.py`). *What happens:* this is the actual proof step, not just a bound calculation. It takes PLEA's and SOAR's *Haemophilus influenzae* data — for which the *complete* result (both positive and negative gene tests) is actually known — and deliberately re-simulates the detection-only scenario (as if only the positive results had been recorded, matching the real-world problem in ATLAS), computes what the Manski bound would have said, and then checks whether the true, fully-known answer actually falls inside that bound. If it does, that's direct empirical proof the bounding method is trustworthy rather than just theoretically appealing.

Budget allocator (`evidence_gate_core/allocator.py`, run with a 200-sample budget in a full pipeline run). Not a validation step — a planning tool. Given a fixed amount of additional lab-testing budget, it uses a standard survey-sampling technique (Horvitz-Thompson estimation) to recommend which specific MIC-value band of isolates would be most valuable to test next, and estimates what confidence interval that additional testing would buy — the "here's what would help" companion piece from Section 3.4.

Export validator (`evidence_gate_core/export_validator.py`). *What happens:* scans every exported file for exactly the failure mode this whole layer exists to prevent — a genotype column (the pipeline checks specifically for the carbapenemase-resistance gene columns NDM, KPC, VIM, IMP, OXA, GES, SPM, and GIM) that has values recorded for positive detections but literally no rows recorded as negative — the detection-only trap explained in Section 3.1. If it finds one, the export is flagged, not silently passed through.

5.3 Phase 3 — Analytics: the original seven-step method, run on the finished table (7 stages)

This is where the original method's seven steps (Section 2.5) actually execute, each stage reading only the finished master table from Phase 1 — never the raw files again — to produce the analytic building blocks for the final published results.

Stage 1 — Descriptive profiling (`step11_descriptive.py`). Computes resistance rates (bacteria) and susceptibility rates (fungi) broken down by organism, drug, drug class, country, year, and body site/specimen source — as Manski bounds, for the same reason as Step 8: any given isolate is only ever tested against some of the drugs in the panel, so a bare rate would understate its own uncertainty. Two real gaps had to be closed to make this possible: body-site/specimen-source data doesn't exist anywhere in the master table, so this step re-joins it directly from the four original raw files (confirmed a 100% match rate); and no drug-class taxonomy exists anywhere in the data or the original brief, so a standard pharmacological classification covering all 30 real drugs was written specifically for this project and saved as its own versioned crosswalk file, `crosswalks/drug_class_crosswalk_v1.csv`, rather than buried in code. Outputs six files (roughly 9,900 to 24,300 rows each) — the largest, `descriptive_fungal_ecv_wt_rate_v1.csv` (24,290 rows), is deliberately kept in a completely separate file from the CLSI-based susceptibility file, because a wild-type/non-wild-type call and a clinical susceptible/resistant call answer genuinely different questions and must never be pooled together.

Stage 2 — Evolutionary layer (`step12_evolutionary.py`). Computes an Evolutionary Distance-to-Failure (how far a country-organism-drug's typical MIC currently sits from the resistance threshold) and an Evolutionary Fitness Score (whether that margin is growing or shrinking year over year, fit as a simple slope across at least 2 qualifying years of

data). About 74% of bacterial isolate-drug rows and about 38.5% of fungal rows resolve a usable anchor point to measure distance from — the shortfall is a real, structural gap: the three echinocandin antifungal drugs (Anidulafungin, Caspofungin, Micafungin) have no numeric threshold of any kind stored anywhere in this pipeline's data, so no trend can be computed for them at all, by design, not by oversight. Outputs four files (2,082 to 9,574 rows).

Stage 3 — Clustering (`step13_clustering.py`) — as already explained in Section 3.5: features are each combination's current burden and its evolutionary trend, clustered separately for bacteria (k=4) and fungi (k=2) using the silhouette-score method. Outputs 640 bacterial and 1,288 fungal country-organism-drug rows.

Stage 4 — External data join (`step14_external_join.py`) — attaches the five external data families from Section 2.4 onto each country-year, via the same ISO3 crosswalk built in Step 1. The actual join success rate varies a lot by variable, and the step reports this honestly rather than implying uniform coverage: life expectancy joins for 217 countries; GBD Socio-demographic Index populates 99 of 99 bacterial country-year rows but only 385 of 413 fungal rows; hospital beds populate 85 of 99 bacterial rows and 350 of 413 fungal rows; Hib3/PCV vaccination coverage is bacteria-only; ESAC-Net antibiotic consumption populates only 21 of 99 bacterial rows (Europe only) and is entirely absent for fungi; GBD lower-respiratory-infection burden is joined but explicitly marked "audit only" — it is never actually used as an input to any regression or ranking, specifically to avoid the impression that it feeds the life-expectancy model. Outputs two country-year tables: 99 bacterial rows, 413 fungal rows.

Stage 5 — Association analysis (`step15_association.py`) — the life-expectancy regression already walked through in Section 3.5: a pooled OLS regression with country-clustered standard errors, n=16 for the bacterial model (flagged for overfitting risk given only 5 countries) and n=269 for the fungal model (36 countries). Alternative model specifications that relax which variables must be non-null (dropping consumption, or dropping vaccination) are also computed and recorded, reaching a larger bacterial sample of up to n=57 — these sit alongside the primary model as a documented sensitivity check, not a replacement for it.

Stage 6 — R&D alignment check (`step16_rd_alignment.py`) — compares Global AMR R&D Hub funding, pro-rated per pathogen and split across matched organisms, against Stage 1's burden numbers, reported as a Spearman rank correlation — with an explicit note in the output that no prior published precedent for this exact comparison method was found, so the comparison itself, not just its result, should be read as exploratory.

Stage 7 — Intervention impact (`step17_intervention.py`) — turns Stage 5's regression coefficients into illustrative scenarios (e.g., the estimated life-expectancy effect of a 10-percentage-point increase in vaccination coverage), plus a separate before/after event-study comparison around actual historical vaccine-introduction dates, computed only where AMR surveillance data exists on both sides of that date. Categories with no usable data (stewardship, diagnostics, water/sanitation/infection-prevention) are explicitly recorded as data gaps rather than scored with an invented number, and fungal vaccination is excluded from the candidate list entirely, by design, since no licensed antifungal vaccine exists to evaluate.

5.4 Phase 4 — Packaging the six deliverables the original brief asked for, then gating them (3 stages)

Step 18 — Section 7 deliverables, ungated (`step18_section7_deliverables.py`). This step does no new modeling of any kind — it only aggregates and ranks what Stages 1–7 already computed, into the six deliverables the original brief's own Section 7 specified: the harmonized dataset manifest, the identifiability ledger, the bacterial/fungal cluster typologies, the bacterial/fungal country risk rankings, the funding-gap summary, and the ranked intervention recommendations. Every ranking rule is written down in a `methodology` column inside the file itself, including two deliberate, documented omissions worth stating plainly: the country risk ranking uses only burden, evolutionary trajectory, and health expenditure — consumption is left out (there's no complete numeric ESAC-Net

series to rank every country on) and vaccination is left out (the original brief's own Output 4 specification never names it as a ranking input).

Step 18b — Gated deliverables (`step18b_gated_deliverables.py`). This is where the quality-gate mechanism from Section 3 actually executes, mechanically, for real: for the cluster typology and country risk ranking files, every row is merged against a lookup of which organism-drug combinations (or, for country rankings, which countries — aggregated as a data-volume-weighted rollup of their underlying organism-drug gates) qualify as `pass`, `bounds_only`, or `withhold`. Any row that isn't `pass` has its numeric score and its rank both wiped to blank — not just the rank, the underlying score too, so a withheld result can't be silently reconstructed by re-sorting on the raw number. Ranks are then recomputed from scratch using only the rows that passed, so rank #1 among published results is always genuinely the best passing result, never a leftover position from a larger unfiltered list. For the intervention recommendations specifically, the gating logic is more elaborate and is worth detailing because it shows the gate catching more than one distinct failure mode at once: a row starts as `pass` by default, then gets withheld if its underlying data status is a known gap (no data, not estimable, not applicable, funding-only with no established life-expectancy link, or excluded by design); withheld if its estimated effect size was already flagged elsewhere in the pipeline as an implausible magnitude likely caused by confounding; withheld if it fails a Bonferroni-corrected significance test applied jointly across every candidate intervention competing for a rank (a stricter bar than checking each one individually, specifically because multiple simultaneous comparisons make an unadjusted single p-value misleadingly optimistic); withheld if it carries any small-sample warning at all — even if its p-value looks clean, because a 16-data-point regression with 10 parameters can produce a deceptively clean-looking p-value purely from overfitting; and withheld if it rests specifically on the Hib vaccination literature sub-measure, which the project's own methodology notes flag as resting on thin outside evidence. Only what survives every one of these checks gets a rank.

Step 18c — Gated country-year panel (`step18c_gated_country_year_panel.py`). Builds one more public deliverable: the country-year life-expectancy panel used for trend charts. It deliberately invents no new gating rule of its own — it just attaches the exact same per-country gate label Step 18b already computed for the country risk ranking. Rows with no real recorded life-expectancy value for that country-year are dropped outright rather than filled in with an interpolated guess.

5.5 Phase 5 — Verification and publish: proving the numbers still hold, then releasing only what's safe (2 stages)

Figure and output verification (`scripts/verify_all_figures.py`). This is not a rerun of the pipeline's own internal Checks — it's an independent, from-scratch recomputation of the specific headline numbers this project relies on (labeled EG-01, EG-02, and so on), reading only the final output files and checking each figure still falls within its expected, previously-established range. If a future data refresh silently shifted a number that no individual step-level Check would catch (because each step only checks its own immediate work, not the eventual downstream headline claim), this is the stage designed to catch it — a full run isn't considered clean unless every one of these checks prints PASS.

Publish to `data/published/` (`scripts/publish_dashboard_data.py`). The very last stage, and the one with the most direct real-world consequence if it went wrong: it copies files from the pipeline's internal `deliverables/` folder into the public, GitHub-tracked `data/published/` folder — but only files matching an explicit public-safe allow-list (every `*_gated_v1.csv` file, the funding summaries, the gating-comparison summary, the identifiability ledger, and a couple of index files). It also maintains an explicit, separate block-list of the *ungated* ranking files (the raw, pre-gate versions of the cluster typology, country risk ranking, and intervention recommendations files) that must never be allowed to reach the public folder — because those files still carry the un-suppressed scores and

ranks that Step 18b specifically wiped for the public versions. This stage is the last line of defense against exactly the kind of accidental leak that would otherwise let someone reconstruct a `withhold`-gated number simply by opening the wrong file. Only after this copy succeeds does the same script assemble `dashboard_bundle_v1.json`, the single file the website actually reads (Section 4).

5.6 What actually gets left behind, in one place

Pulling every exclusion made across the whole 30-stage run into one list, since "what gets left behind" is exactly the kind of question a careful reviewer asks: 613 isolates excluded for `Evaluable=N` (SOAR 207965 only, kept in the registry and logs, excluded from the master table and every downstream denominator); 29 isolates excluded for organism reasons (contaminants, no-growth, cross-domain fungal, also SOAR 207965 only); 0 duplicate isolates actually removed (candidates are logged for human review, never auto-deleted); a large share of bacterial and fungal isolate-drug rows that get an interpretable resistance category but are gated `bounds_only` or `withhold` rather than a bare percentage (Section 3.3's chart); and, at the very final publish step, every ungated ranking file — deliberately excluded from the public repository entirely, on every single run, by the block-list described above.

---

6. How it runs — a full operational walkthrough

6.1 Running the pipeline

One command, no extra options needed:

```
python analysis/run_all.py
```

This tells Python to look inside `analysis/src/` for the pipeline's code, then calls a function named `run_full_pipeline()`. Its own internal documentation states the intent in one line: *"Harmonize → integrity → analytics → brief §7 deliverables → verification."* Every single run writes a file called `runs/<run_id>/pipeline_run_manifest_v1.json` — a permanent record of exactly what that run did, so any past run can be looked up later and nothing is undocumented.

The full 30-stage breakdown — every stage's inputs, transformations, exclusions, and outputs — is Section 5 above. In short: 13 preprocessing stages + 5 integrity stages + 7 analytics stages + 3 deliverable stages + 2 verification stages = **30 stages total**, the registry's own exact count, not a rounded estimate. A pipeline run isn't considered clean unless the verification phase (Section 5.5) prints PASS on every check.

6.2 Publishing the results

Publishing happens automatically at the end of a full pipeline run. It can also be re-run on its own, without re-running the whole 30-stage pipeline, using:

```
python analysis/scripts/publish_dashboard_data.py
```

This does two things: (1) it (re)writes the gated CSVs and the two JSON files (`dashboard_bundle_v1.json`, `dataset_status_v1.json`) into `data/published/`, and (2) it copies those same two files into `dashboard/public/data/published/`, which is the copy the website actually reads. If pipeline logic changes but the dashboard doesn't seem to reflect it, the most likely cause is forgetting this second copy step — both `dashboard_bundle_v1.json` files must be identical.

6.3 Running the dashboard website

```
cd dashboard
npm install
npm run dev
```

(`npm` is the standard tool for installing and running JavaScript/web projects; `npm install` downloads everything the website's code depends on, and `npm run dev` starts a local test copy of the website on your own computer.) The dashboard's own configuration doesn't set a fixed port number — in this environment, that resolves to **`http://localhost:8080/`** (confirmed from an actual run this session), not the `:5173` the dashboard's own README currently states — if you're pointing a teammate at a locally running copy, use whatever address your own terminal actually prints.

To produce a deployable, production-ready copy of the site:

```
npm run build
```

No login credentials or special configuration are required just to see the core AMR results — the login system (Supabase) is only needed for the account/admin features described in Section 8.3. As long as `dashboard_bundle_v1.json` is present, the core dashboard works entirely on its own.

---

7. What gets generated — every report and file, explained

7.1 The gated CSVs — the audit-trail layer

Nine spreadsheet files, one per major result, each with a `quality_gate` column spelling out `pass`, `bounds_only`, or `withhold` per row: the bacterial and fungal cluster typologies, the bacterial and fungal country risk rankings, the intervention recommendations, the funding-gap summary, the Hub funding composition breakdown, the identifiability ledger, and a gating-comparison summary. These are the files a teammate, judge, or reviewer would open directly in Excel to check a specific claim without touching the dashboard at all.

7.2 `dashboard_bundle_v1.json` — the dashboard's single data source

One JSON file, split into named sections, each one feeding a specific part of the website:

Bundle section	What it feeds
`countryRiskBacterial` / `countryRiskFungal`	The world risk map and the country ranking table
`countryYearBacterial` / `countryYearFungal`	Country-by-year trend lines
`clusterTypologyBacterial` / `clusterTypologyFungal`	The organism-drug typology explorer
`interventions`	The intervention policy table
`fundingGap` / `fundingByYear` / `hubFundingComposition`	The funding-gap explorer
`identifiabilityLedger`	The "why can't this row have a percentage" explanations
`gatingComparison`	The evidence-gate summary counts (Section 3)
`q2DriverSummary`	The life-expectancy driver breakdown (answering Q2)

Bundle section	What it feeds
`associationSensitivity`	Model-reliability and robustness notes
`deliverablesIndex`	The reports/exports page's list of downloadable sections
`pipeline_run`	The `run_id` and timestamp shown in every page footer

7.3 The Vivli submission report

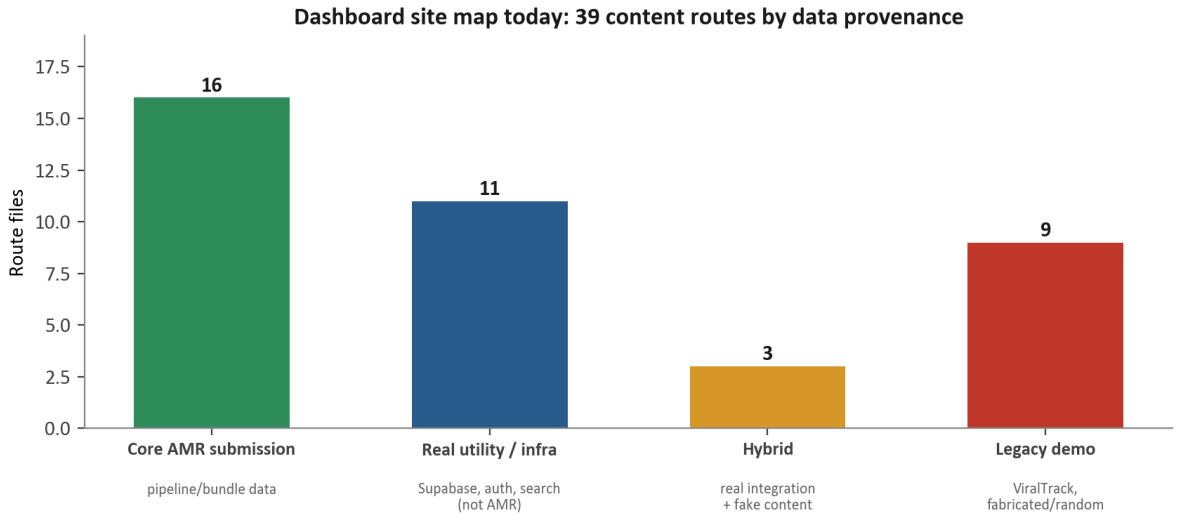
`docs/VIVLI_SUBMISSION_REPORT_v1.md` is the official 5-page report submitted to Vivli (5 pages is their stated maximum). It covers the Abstract, Objectives (the same five guiding questions from Section 2.2), Methods, Results, Impact, and Limitations, with two embedded charts. Every sentence in it names a specific published file it came from. Section 10 below is a condensed version of its current Results section.

---

8. The dashboard — every route, what it shows, and what it's for

A quick vocabulary note first: a **"route"** is just one distinct page/URL of the website (e.g. the address that shows the country map is a different route from the one that shows funding). The **"tab bar"** is the row of clickable page names at the top of the site; not every route is in it. **Supabase** is an outside service this project uses for login accounts, file storage, and an admin console — it's a real, working piece of infrastructure, but it's separate from the AMR science described above.

The dashboard has 43 route files in total. Four of them are pure infrastructure with no visible page of their own — the root app shell (`__root.tsx`, which sets up shared background plumbing but shows no content itself), two routes that just redirect elsewhere (`/auth` → `/login`, `/admin` → `/admin/dashboard`), and the shared wrapper around all admin pages. The remaining 39 fall into four groups:



Source: direct read of all 43 files in dashboard/src/routes/ (4 are infra-only shells/redirects, excluded above)

8.1 How the page layout and navigation actually work

Every real page is wrapped in a shared layout component that renders, top to bottom: a header (logo, search box, links to two utility pages, a notification icon), a scrolling ticker, a row of tabs (on most pages), the page's own

content, then a footer stamped `Pipeline {run_id} · bundle {generated_at}` — the same freshness stamp mentioned in Section 4.

The main tab row has 11 tabs, visible on every page except four that intentionally hide it: **Overview** (`/`), **Risk Map** (`/alerts`), **Countries** (`/countries`), **Resistance** (`/pathogens`), **Evolution** (`/lineages`), **Life Exp.** (`/epidemiology`), **Funding** (`/marketplace`), **ML** (`/forecasting`), **Ingest** (`/ingest`), **Reports** (`/reports`), **Methods** (`/methodology`).

A keyboard shortcut (Ctrl/Cmd+K) opens a command palette with three more pages not in the tab row: `/policy` (Intervention Policy), `/assistant` (AI Assistant), `/api-docs` (API Documentation).

That's **14 routes** a typical visitor finds through ordinary browsing. The other 25 exist only behind a specific in-page link or by typing an exact web address directly — 14 of the 43 files have *no* link pointing to them anywhere in the app at all.

8.2 Core AMR submission routes — real pipeline data, described one by one

- `/` **(Overview)** — the homepage. Summary statistic tiles and charts, all pulling from the real published bundle.
- `/alerts` **(Risk Map)** — a world map of AMR risk, explicitly labeled as refreshing from the published bundle every 60 seconds. (This is what "live" means throughout this dashboard: a periodic re-check of the same static file, never a live-streaming database — worth remembering everywhere the word "live" appears.)
- `/countries` **(Countries)** — a per-country trend table.
- `/pathogens` **(Resistance)** — the "Resistance Explorer." Notably, if the real published data ever fails to load, this page explicitly shows "no demo fallback for this view" instead of quietly showing fake numbers in its place — the opposite failure mode from the legacy pages in Section 8.4.
- `/lineages` **(Evolution)** — the "Evolution Explorer," showing the trajectory/fitness-slope work from Section 2.5, step 2.
- `/epidemiology` **(Life Exp.)** — the life-expectancy association view — the dashboard's version of Section 3.5's chart.
- `/marketplace` **(Funding)** — despite the URL's name, this is the "Funding Gap Explorer" — the Q3 funding-vs-burden comparison. The route's name is a leftover label; the content is real.
- `/forecasting` **(ML)** — shows real published data, but its own on-page text says the main navigation has moved on to Overview/Evolution/Methods and this page is kept only for old bookmarked links. It's still in the main tab row today even though its own text says it's been superseded — worth knowing if a teammate asks why it feels slightly out of place.
- `/reports` **(Reports)** — four downloadable report packages (an executive policy brief, a scientific report, a demo slide deck, and raw data exports), all built from real published sections, exportable as PDF, Word, spreadsheet, or JSON. One format (PowerPoint) is honestly unfinished: choosing it downloads a JSON file instead, with a message explaining PowerPoint export isn't built yet — rather than silently producing a broken file.
- `/methodology` **(Methods)** — the dashboard's own internal version of what Sections 3, 5, and 7 of this document explain: the pipeline stages, the gate outcome counts, the model-reliability notes.
- `/policy` (command palette only) — the intervention recommendations table from Section 3.3, showing every candidate row's gate status and, for withheld rows, its specific reason.
- `/api-docs` (command palette + header link) — real technical documentation of the actual bundle file's structure, for a developer who wants to read it directly.

- `/about` — a static introduction page with the project's real scale figures (57 countries, roughly 34,000+ isolates, 4 cohorts) and a description of what the project actually is. This is the very first page every visitor sees after logging in.
- `/team` — the real Vivli 2026 submission team: 5 named people with their affiliations.
- `/news` (no menu entry, reachable only by direct link) — despite the name, this is an honest "Evidence Gate & Pipeline Activity" feed showing real gate statistics and pipeline status — its own page text explicitly states there's no external news wire and nothing here is simulated.
- `/partnerships` (no menu entry) — the real data-sources and acknowledgments list: the four cohorts plus every external dataset from Section 2.4, named properly.

8.3 Real, but non-AMR: utility and infrastructure pages

These pages are genuinely functional, not fake — but they're the web app's plumbing, not part of the AMR science: the login and account-approval pages, a full admin console (user approval, role management, a security audit log — all backed by the real Supabase service), a search page (which searches both an internal database and a genuinely live external scientific-literature search), and a private per-user file-upload area.

One page, `/ingest`, sits in the main tab row but is a special, clearly-labeled case: it lets you preview a CSV file in your browser, but its own on-page message says plainly that no actual processing pipeline is connected to it yet — an honestly-labeled placeholder, not fake data pretending to be real.

8.4 Leftover pages from an earlier, different product

Before this became a Vivli AMR statistical submission, this same codebase was originally being built as a *different* product — a general infectious-disease outbreak surveillance tool covering things like viral variant tracking and lab-network monitoring, unrelated to antimicrobial resistance. That earlier direction is why nine of the 43 routes still exist today containing invented or randomly-generated example content that has nothing to do with this project's actual AMR pipeline: a training-course page, a "biosecurity threats" list, a static emergency-response page, a page that generates random signal values on the spot, a comparison tool built on fake data, a hardcoded "active signals" page, a notifications settings page built around unrelated disease names, a large interactive family-tree-style viewer built around one made-up example tree, and a 3D molecule viewer wrapped around a fixed, unrelated example list (the underlying 3D-viewer technology itself is a genuine, live external tool — only the example list around it is not relevant here).

Most of these nine carry a clearly visible on-page banner stating plainly that the content is illustrative and not part of this submission — but five of the nine do not yet have that banner. A visitor who reaches one of those five by typing a direct link would see no on-page warning that the numbers are fabricated. None of the nine are reachable through the main tab row or the keyboard-shortcut menu, which limits — without fully eliminating — how likely an ordinary visitor is to land on one by accident.

Two further routes are a mix of both: one real, working search feature (a real, live external medical-literature search, and a real internal database query) sitting on the same page as some invented illustrative charts; the other, a real internal database table shown at the bottom of a page whose upper charts are randomly generated.

8.5 What to tell a teammate in one sentence

Every page reachable through the normal tab row or the keyboard-shortcut menu (14 pages total) is either real AMR pipeline data or honest documentation of it, with two labeled exceptions (one page says it's an unfinished stub, one shows real data behind an outdated "this has moved" notice). Anything found only by clicking deeper — a

footer link, a search result, or a typed-in address — may be leftover example content from an earlier, unrelated product, and about half of those pages say so clearly on-screen and half don't yet.

---

9. How today's build compares to the original idea

Going back to the original brief's own tracking table (`internal/brief/AMR_LIFE_EXPECTANCY_UNIFIED_PROJECT_BRIEF.md`), here's how each of the five original questions maps to what exists today — using the brief's own language for what's fully built versus what has a known, acknowledged gap:

Question	What the original plan called for	Status today
Q1 — resistance vs. life expectancy	Country risk ranking combining burden, trajectory, consumption, and health-system capacity	Built and gated (Section 3.3): 29/30 bacterial countries ranked; fungal ranking withheld entirely, honestly, for insufficient data
Q2 — what drives it	Regression including consumption, vaccination, and hospital-exposure data	Built for the data that exists; antimicrobial consumption data only ever covered Europe (a limit the brief itself flagged in advance, Section 2.6); no hospital-acquired-exposure dataset has ever been available to join
Q3 — R&D funding vs. burden	Funding-gap summary, Hub vs. surveillance burden	Built and published in full (Section 10)
Q4 — early-trajectory combinations	Cluster typology separating high-burden from high-trajectory	Built and published in full
Q5 — best intervention category	Ranked intervention list with estimated life-expectancy gains	Built, but every candidate is currently gated `withhold` (Section 3.3) — the mechanism works exactly as designed; the underlying data simply isn't strong enough yet to support a confident ranking

The bottom line: the project did not drift from the original idea — the original five questions, the four cohorts, and the seven-step method are all still exactly what's running today. What changed is that an integrity layer (Section 3) was deliberately added on top, exactly as the original brief itself called for, so that the answers to those five questions are only shown when the data can actually support them — and the two places where the honest answer today is "not yet" (fungal country ranking, intervention ranking) are reported as such rather than papered over.

---

10. Current results, question by question

Condensed from `docs/VIVLI_SUBMISSION_REPORT_v1.md`, verified against pipeline run `20260720T144743`:

Q1 — Which resistance profiles co-occur with the lowest life expectancy? Bacterial typology publishes all 640 of 640 possible combinations (387 "moderate," 92 "high trajectory," 92 "high burden," 69 both-high). Bacterial country risk ranking passes for 29 of 30 countries, led by Singapore, Cambodia, Kenya, India, and the UAE. Fungal typology publishes only 107 of 1,288 combinations (8.3%) after gating, and fungal country-level ranking passes for none of 43 countries — the project reports that limit rather than inventing a number.

Q2 — What drives the relationship? In the fungal model (269 country-years across 36 countries, explaining 63.8% of the variation in life expectancy), the GBD Socio-demographic Index dominates by a wide margin and is highly statistically significant. Health spending and hospital-bed capacity are not statistically distinguishable from zero once the country-clustering correction from Section 3.5 is applied. The bacterial model (only 16 data points across 5

countries) is too small to support any coefficient-level claims and isn't used to make them. Antibiotic-consumption data only covers 21 European bacterial rows; there is still no antifungal-consumption or hospital-exposure dataset available anywhere.

Q3 — Where does R&D funding concentrate relative to burden? All four bacterial organisms studied show a burden share well above their matched funding share — for example, *K. pneumoniae* accounts for 32.3% of the studied burden but only 1.7% of matched Hub funding. Of the Hub's entire $18.84 billion tracked portfolio, only 2.1% is attributable to Sub-Saharan Africa, versus 97.0% elsewhere.

Q4 — Which combinations are early-warning candidates? 92 of 640 bacterial combinations, and 30 of the 107 gated fungal combinations, are classified "high trajectory" without yet being "high burden" — these are the project's proposed early-intervention watchlist, the combinations worth acting on before they become a crisis rather than after.

Q5 — Which intervention would yield the largest life-expectancy gain? None of the 11 candidate intervention rows currently clear the integrity bar for a public ranking. The dashboard's policy page shows this honestly: an empty ranked list, with each row's specific reason listed (too small a sample, a data gap, funding-only data with no life-expectancy link established, or — for fungal vaccination specifically — excluded by design because no licensed vaccine exists for it), rather than forcing a ranking the underlying data can't support.

---

11. Current known limitations

Stated as current facts about the project as it exists today, not as defects awaiting a fix:

- Most organism-drug combinations still cannot support a single confident number — roughly 1 in 5 fungal isolate-drug rows have no published breakpoint standard to classify against at all.
- The bacterial life-expectancy model's sample (16 data points, 5 countries) is too small to support coefficient-level claims; it exists in the pipeline but isn't used to make them.
- Antibiotic-consumption data only exists for Europe; no hospital-acquired-exposure dataset has ever been available to join in, anywhere.
- The fungal side of the project rests on a single surveillance cohort (SENTRY) with no second fungal cohort available to cross-check it against — unlike bacteria, which draws on three independent SOAR rounds.
- A dashboard chart labeled "resistance over time" shows a genuine per-year, cross-country average — not tracking of the same individual patients over time — which is a real but coarser signal than the label alone might suggest.
- "Live" language throughout the dashboard (the ticker, the footer, the 60-second risk-map refresh) means a periodic re-check of the same static published file, never a real-time streaming feed — worth remembering wherever that word appears.
- The dashboard's own setup instructions currently state the local test server runs on port 5173; in practice it starts on port 8080 (Section 6.3) — noted here so nobody chases the wrong web address.
- Of the nine leftover example pages from the project's earlier, unrelated product direction (Section 8.4), five don't yet carry the on-page notice that the other four have, so a visitor reaching one directly has no on-screen warning that it isn't real.

---

12. Glossary — every term used in this document, A to Z

Term	Plain-language meaning
AMR	Antimicrobial Resistance — microbes evolving so that drugs which used to kill them stop working
API	A live connection where software asks a server a question and gets a fresh answer in real time; this project deliberately avoids one for its AMR data
ATLAS	A separate, much larger surveillance dataset used only to stress-test the integrity method at scale — not one of the four core cohorts
Bounds / Manski bounds	A published range ("somewhere between X and Y") used instead of a single guessed number, when the data can't support a precise point estimate
Breakpoint	An official published threshold: below it, a drug is considered to work (Susceptible); above it, it's considered not to work (Resistant)
Bundle (`dashboard_bundle_v1.json`)	The single packaged data file the dashboard website reads to show everything it shows
Cluster / clustering	Automatically sorting data points into groups based on how similar they are to each other
CLSI	Clinical and Laboratory Standards Institute — publishes the reference thresholds used for fungi
Cohort	One surveillance program with its own testing sites and reporting format; this project combines four
Cluster-robust standard error	A statistical correction that accounts for multiple data points from the same country not being fully independent of each other
ECV	Epidemiological Cutoff Value — a softer classification standard used for fungi when no formal breakpoint exists yet
ESAC-Net	A European network reporting how much antibiotic medicine is used per country per year
EUCAST	European Committee on Antimicrobial Susceptibility Testing — publishes the reference thresholds used for bacteria
GBD / SDI	Global Burden of Disease study; SDI (Socio-demographic Index) is its composite score of a country's income, education, and fertility level
Gate / quality_gate	The label (`pass`, `bounds_only`, or `withhold`) attached to every published result, describing how confident the project is in that specific number
ISO3	The standard 3-letter country code (e.g. `KEN`, `USA`) used to match the same country across different datasets
Isolate	One single bacteria or fungus sample taken from one patient
JSON	A plain-text file format for storing structured data, readable by both humans and software
Life expectancy	The average number of years a newborn in a given country is expected to live; this project's outcome variable
MIC	Minimum Inhibitory Concentration — the lowest drug amount that stopped a microbe from growing in a lab test
Pipeline	The sequence of scripts that clean, analyze, and publish the data, run one after another
p-value	A statistic describing how likely a result could have occurred by chance alone; a small p-value (conventionally below 0.05) is called "statistically significant"

Term	Plain-language meaning
R^2	A 0-to-1 measure of how much of the variation in an outcome (here, life expectancy) a statistical model explains
Resistant / Susceptible / Intermediate (R/S/I)	The three possible outcomes of comparing a lab result to a breakpoint
Route	One distinct page/web address on the dashboard website
SENTRY	The single fungal (antifungal) surveillance cohort this project uses, covering 2010–2024
Silhouette score	A statistic measuring how cleanly a clustering result separates its groups; higher is better
SOAR	Survey of Antibiotic Resistance — three separate bacterial surveillance rounds this project combines, named by their internal round codes: 201818, 201910, and 207965
Supabase	An outside service this project uses for login accounts, an admin console, and file storage — separate from the AMR pipeline data
Wild-Type / Non-Wild-Type (WT/NWT)	The fungal equivalent of Susceptible/Resistant, used where a formal clinical breakpoint doesn't exist

---

13. Where to look for more

- **Original project brief:** `internal/brief/AMR_LIFE_EXPECTANCY_UNIFIED_PROJECT_BRIEF.md`
- **Pipeline stage registry (ground truth for all 30 stages):** `analysis/src/pipeline_registry.py`
- **Preprocessing glossary and cohort detail:** `analysis/README.md`
- **Publish mechanics:** `data/published/README.md`
- **Dashboard architecture, design choices, known gaps:** `dashboard/README.md`
- **Full submission report:** `docs/VIVLI_SUBMISSION_REPORT_v1.md`
- **Every gated result file:** `data/published/*_gated_v1.csv`
- **The dashboard's own data bundle:** `dashboard/public/data/published/dashboard_bundle_v1.json`